Aufgabe: Abstraktion (Abstract Classes and Methods)

Für: Michael **Referenz:** W3Schools – C# Abstraction **Bearbeitungszeit:** ca. 45–60 Min. **Voraussetzung:** Diese Aufgabe baut direkt auf deiner **Vererbungs-** und **Polymorphie-Aufgabe** (`Fahrzeug` , `Auto` , `Motorrad` , `Fahrrad`) auf. Neu ist nur **Abstraktion**. **Lernziel:** Am Ende verstehst du, was eine abstrakte Klasse/Methode ist, kannst erklären, wofür sie gut ist, und kannst sie selbst in einer eigenen Klassenhierarchie anwenden.

---

1. Was du schon kannst (kurzer Rückblick)

- **Vererbung:** `Auto : Fahrzeug` – eine abgeleitete Klasse erbt von einer Basisklasse
- **Polymorphie:** `virtual` (Basisklasse: "darf überschrieben werden") + `override` (Kindklasse: "wird ersetzt")
- **`List<Fahrzeug>` mit `foreach` :** gemischte Objekte durchlaufen, jedes verhält sich passend zu seinem eigenen Typ

Eine Lücke ist dir dabei vielleicht schon aufgefallen: Bisher **konnte** man `Hupen()` überschreiben – man **musste** aber nicht. Nichts hätte dich gehindert, ein `Fahrzeug` -Objekt direkt zu erzeugen (`new Fahrzeug(...)`), obwohl "ein Fahrzeug an sich" eigentlich nie konkret vorkommt. Genau diese zwei Lücken schliesst Abstraktion.

2. Neu: Abstraktion

Lies zuerst das W3Schools-Kapitel: **C# Abstraction**.

Die Kernidee laut W3Schools: *"Data abstraction is the process of hiding certain details and showing only essential information to the user."* Erreicht wird das entweder mit **abstrakten Klassen** oder mit **Interfaces** (Interfaces kommen erst im nächsten Kapitel).

Zwei neue Begriffe: - **Abstract class:** *"a restricted class that cannot be used to create objects"* – man kann **kein** Objekt direkt davon erzeugen, nur davon erben. - **Abstract method:** *"can only be used in an abstract class, and it does not have a body. The body is provided by the derived class."* – die Basisklasse gibt nur die Signatur vor, keine Umsetzung.

Das W3Schools-Beispiel (Weiterentwicklung von Polymorphie)

W3Schools nimmt genau das `Animal` -Beispiel aus dem Polymorphie-Kapitel und macht es abstrakt:

```
abstract class Animal
{
  public abstract void animalSound();  // keine geschweiften Klammern, kein Body!
  public void sleep()                  // normale Methode, ganz wie gewohnt
  {
    Console.WriteLine("Zzz");
  }
}

class Pig : Animal
{
  public override void animalSound()
  {
    Console.WriteLine("The pig says: wee wee");
  }
}
```

```
Animal myObj = new Animal();  // Fehler! "Cannot create an instance of the abstract class"
Pig myPig = new Pig();        // funktioniert
myPig.animalSound();
myPig.sleep();                // geerbt, ganz normale Methode
```

Der entscheidende Unterschied zu dem, was du bei Polymorphie gemacht hast:

	`virtual` (Polymorphie-Aufgabe)	`abstract` (diese Aufgabe)
Body in der Basisklasse?	Ja, eine "Standard"-Umsetzung	Nein, gar keiner
Überschreiben in der Kindklasse	optional	**Pflicht**
Objekt der Basisklasse selbst erzeugen?	erlaubt	**verboten**

---

3. Deine Aufgabe: `Fahrzeug` wird abstrakt

Schritt 1 – Fehler bewusst provozieren

Bevor du etwas änderst: Erzeuge in deinem bestehenden Code testweise ein reines `Fahrzeug`-Objekt:

```
Fahrzeug irgendeins = new Fahrzeug("???");
```

Das funktioniert aktuell noch anstandslos. **Leitfrage:** Ergibt es inhaltlich überhaupt Sinn, ein "Fahrzeug an sich" zu besitzen, ohne dass es ein Auto, Motorrad oder Fahrrad ist?

Schritt 2 – `Fahrzeug` zur abstrakten Klasse machen

Ändere den Klassenkopf:

```
abstract class Fahrzeug
{
    // dein bestehender Code (Feld, Property, Konstruktor)
}
```

Kompiliere neu und erzeuge wieder `new Fahrzeug("???")`.

Vorhersage zuerst: Was, glaubst du, passiert jetzt? Notiere deine Vermutung, dann testen und die Fehlermeldung genau lesen.

Schritt 3 – `Hupen()` zur abstrakten Methode machen

Ändere in `Fahrzeug`:

```
public abstract void Hupen();   // kein Body, endet mit Semikolon!
```

Kompilier-Check: Was passiert jetzt mit `Auto`, `Motorrad` und `Fahrrad`, falls eine dieser Klassen `Hupen()` bisher **nicht** überschrieben hat? Lies die Fehlermeldung, falls eine erscheint, und behebe sie.

Schritt 4 – Eine normale (nicht-abstrakte) Methode ergänzen

Damit du den Unterschied direkt im selben Code siehst, ergänze `Fahrzeug` um eine ganz normale, fertige Methode:

```
public void Warten()
{
    Console.WriteLine("Wird gewartet...");
}
```

Ruf `Warten()` auf einem `Auto`-Objekt auf, **ohne** dass `Auto` diese Methode selbst schreibt.

Leitfrage: Warum funktioniert `meinAuto.Warten()`, obwohl `Auto` diese Methode nirgends erwähnt – während `Hupen()` in jeder Kindklasse zwingend stehen musste?

Schritt 5 – Alles zusammen testen

Nutze deine `List<Fahrzeug>` aus der Polymorphie-Aufgabe weiter:

```
List<Fahrzeug> meineFahrzeuge = new List<Fahrzeug>();
meineFahrzeuge.Add(new Auto("Ford", "Mustang"));
meineFahrzeuge.Add(new Motorrad("Honda", 650));

foreach (Fahrzeug f in meineFahrzeuge)
{
    f.Hupen();
    f.Warten();
}
```

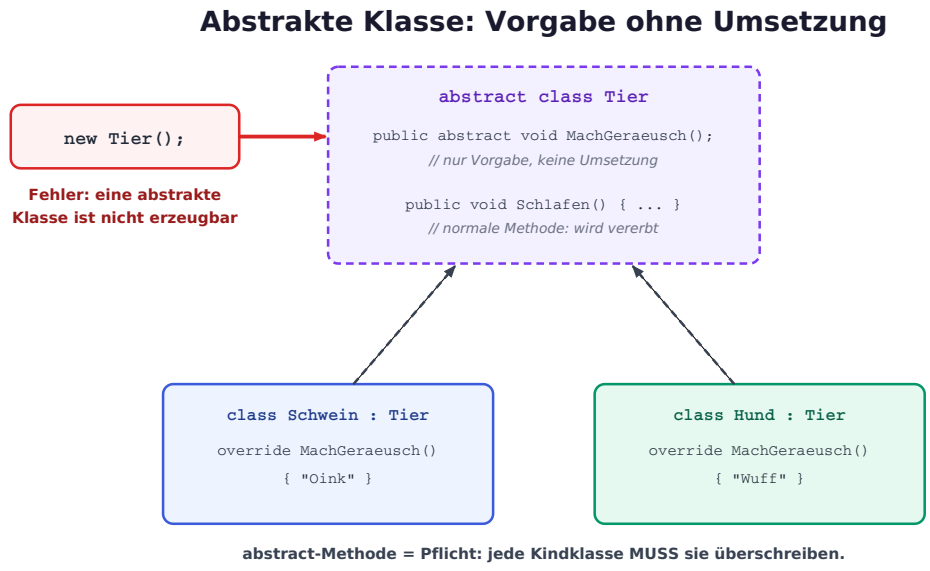
Vorhersage zuerst: Was gibt das aus? Wie viele Zeilen, welcher Inhalt?

---

4. Stolperstein-Check

- ☐ Hast du `abstract` sowohl vor `class Fahrzeug` als auch vor `void Hupen()` geschrieben? (Beides zusammen macht die Methode wirklich abstrakt.)
- ☐ Hat `Hupen()` in der Basisklasse **keine** geschweiften Klammern `{ }`, sondern endet direkt mit `;`?
- ☐ Überschreibt **jede** Kindklasse (`Auto`, `Motorrad`, `Fahrrad`) `Hupen()` mit `override`? (Fehlt eine, meldet der Compiler das sofort – das ist Absicht, nicht ein Fehler in deinem Code.)
- ☐ Hast du versucht, `new Fahrzeug(...)` zu erzeugen, und die Fehlermeldung wirklich gelesen (nicht nur weggeklickt)?

5. Bild zur Orientierung



Abstrakte Klasse

Leitfragen zum Bild: - Warum kann man von der abstrakten Klasse kein Objekt erzeugen? - Was ist der Unterschied zwischen der abstrakten Methode und einer normalen Methode wie `Schlafen()` / `Warten()` im selben Bild? - Was müssen die Kindklassen zwingend tun?

---

6. Transferaufgabe (eigenständig, ohne Vorlage)

Baue **ohne Hilfe** eine zweite, kleine abstrakte Hierarchie zu einem Thema deiner Wahl (z.B. `Form` mit `Kreis` / `Rechteck` und einer abstrakten Methode `BerechneFlaeche()`, oder `Instrument` mit `Gitarre` / `Klavier` und `SpielTon()`). Anforderungen:

- Eine `abstract class` mit mindestens einer abstrakten Methode **und** mindestens einer normalen Methode
- Mindestens zwei Kindklassen, die die abstrakte Methode überschreiben
- Ein Test, der zeigt, dass man von der Basisklasse kein Objekt erzeugen kann (Kommentar im Code reicht, muss nicht einkompiliert werden)
- Eine `List<BasisTyp>` mit `foreach`, die polymorph beide Kindklassen behandelt

Das ist der eigentliche Beweis, dass du Abstraktion nicht nur nachgebaut, sondern verstanden hast.

---

7. Abschluss-Fragen (mündlich beantworten)

1. Was ist der Unterschied zwischen einer abstrakten Methode und einer `virtual`-Methode?
2. Warum kann man von `Fahrzeug` (jetzt `abstract`) kein Objekt mehr erzeugen?
3. Was passiert, wenn eine Kindklasse eine abstrakte Methode **nicht** überschreibt?
4. Kann eine abstrakte Klasse auch ganz normale, fertige Methoden enthalten? Nenn ein Beispiel aus deiner eigenen Fahrzeug-Klasse.
5. Laut W3Schools gibt es einen zweiten Weg, Abstraktion zu erreichen. Welchen?

8. Weiterführend

Der nächste Schritt: **C# Interface**. Interfaces lösen eine Einschränkung, auf die du dabei stossen wirst: Eine Klasse kann nur von **einer** Basisklasse erben – aber vielleicht möchtest du, dass eine Klasse mehrere unabhängige Fähigkeiten gleichzeitig hat.